

The Grader: Design From the Literature

5 October 2026 · How to grade an in-situ dependency removal so that no cheat we have found survives

The one idea everything rests on

Every prior grader for repository changes has the same weakness: it has no reference for what the code *should* do, so it falls back on the project's tests, and tests are a sample.

Our task is different in one way that nobody has exploited. **The thing being removed is a working reference implementation of exactly the behaviour that must be preserved.** The library is right there. It has been running inside this project's tests for years. We know, for every call the tests make, what it returned.

That turns an unsolvable problem into a solved one. "Is this rewrite correct?" has no general answer. "Does this rewrite return what the library returned on these inputs, and on these ten thousand neighbouring inputs?" does. The field that answers it is forty years old. We are not inventing a grader; we are recognising that our task hands us the oracle that other tasks lack.

Three literatures combine into the design: patch-overfitting detection from program repair, carving and record-replay testing, and extreme mutation analysis. Each solves one of our three open problems.

The three problems, and who already solved them

Problem 1: a hollow replacement passes the tests (the stub)

Our fakes that returned nothing passed CI 8 times in 62, and no install-level check saw them. Sushane's smarter fake, filler logic in an internal module with no name reuse, is structurally identical to a real rewrite in 79 of 80 cases. **Static analysis cannot solve this, and the literature agrees.**

This is the *patch overfitting* problem, which automated program repair has fought since 2015 ([Smith et al., FSE 2015](#)). A patch passes the test suite but is wrong. The surveys ([Patch Correctness Assessment: A Survey, TOSEM 2024](#)) classify the solutions, and the dynamic ones map directly onto us:

APR technique	What it does	Our equivalent
DiffTGen (Xin & Reiss, ISSTA 2017)	Generate inputs that distinguish patched from reference program	Generate inputs that distinguish the rewrite from the library
Opad (Yang et al., FSE 2017)	Fuzz existing tests; implicit oracles (crash, memory)	Fuzz the test-provided inputs through both
PATCH-SIM (Xiong et al., ICSE 2018)	Passing tests should behave the same before and after a correct patch	Execution traces at the call boundary should match
PatchDiff (Wang, Pradel, Liu, ICSE 2026)	Differential testing of two patches finds 29.6% behavioural divergence on SWE-bench	Differential testing of rewrite vs library

The APR field has one handicap we do not: their reference is a single human patch that may itself be imperfect. Ours is the library, which is canonical by definition. **So the APR machinery works better for us than it did for them.**

Problem 2: the tests only constrain the inputs they happen to use

A stub that hardcodes the test inputs passes. So does a rewrite that is correct on ASCII but wrong on Arabic, as our `wcwidth` case was. The tests are a sample of the input space; the library is correct on all of it.

Extreme mutation analysis quantifies exactly this. Replace a function's body with `return <constant>` and rerun the tests. If they still pass, the function is *pseudo-tested*: executed, but its result never checked. Descartes ([Vera-Pérez et al., EMSE 2019](#)) found 1 to 46 percent of methods pseudo-tested across 21 mature projects. Our stub result is this phenomenon observed from the other side: a stub survives exactly when the function it replaces is pseudo-tested.

This gives us something new: a **per-instance, per-function confidence score** rather than a binary gate. For each function the agent wrote, we can say whether the test suite would have noticed if it were hollow.

Problem 3: we need an oracle the agent cannot see

Everything the agent can see, it can overfit. SpecBench's remedy is held-out tests the agent never sees, and the hacker-fixer paper ([arXiv 2606.08960](#)) shows verifiers must be hardened adversarially or they are gamed. A grader built only from the visible test suite is a target.

Carving solves this. Elbaum, Dwyer and colleagues ([ICSE 2006, TSE 2009](#)) record program state at a method's entry and exit while system tests run, then replay that recorded state as a unit test with a built-in oracle. Meta does this in production today ([observation-based test generation, 2024](#)). The recorded behaviour is a *second* test suite derived from the first, and the agent never sees it because it is generated at grading time from the gold run.

The grader

Eight gates. A change is accepted only if it passes all of them. G1 to G3 and G5 are install-level and already built. G4 is the heart of the design. G6 to G8 are deterministic additions.

Phase 0: record the reference, before the agent runs

Done once per instance, at benchmark construction, never visible to the agent.

1. Install the gold environment, library present.
2. Identify the **usage sites**: every project function that calls into the library. We already compute these from the import graph.
3. Wrap each usage site with a recorder (`sys.setprofile` or decorator injection in a scratch copy). Run the full test suite. For every invocation, record arguments and return value, serialised; for unserialisable objects, record a structural digest.
4. Also record at the **library boundary**: every call into the library's API, with arguments and results. This is the carved reference.
5. Store both recordings as the instance's **hidden oracle**.

The recording is at the usage site, not only the library boundary, for a reason. The agent's replacement will not have the library's function names, so library-boundary recordings cannot be replayed against it directly. The project's own functions keep their names across the change. Replaying at the usage site sidesteps the mapping problem entirely.

G1 Declaration gone

Package absent from the manifest and from `pip freeze` of the resolved environment. Catches *hide*.

G2 Tests pass

The project's own suite, in a container. What DI-Bench already does.

G3 No vendored copy

Winnowing fingerprints ([Vendetect](#), [Trail of Bits 2025](#)) of every added or modified file against the library's distributed source. Winnowing survives renamed identifiers, stripped comments and reformatting, where our current token-Jaccard check does not. Our audit found G3 only inspected newly added files; this version inspects modified files too.

G4 Behaviour preserved: the differential oracle

Three layers, each catching what the previous one lets through.

G4a Carved replay. Replay every recorded usage-site invocation against the agent's version. Compare returns with a normalised equality (sorted dict keys, float tolerance, type-aware). Any divergence on an input the tests actually exercised is a definite behavioural change. This catches the `wcwidth` case: the simplified width function returns a different value for an Arabic string the tests fed it. It is PATCH-SIM's principle at the function boundary.

G4b Differential fuzzing. For each usage site, derive Hypothesis strategies from the *types and value distributions* of the recorded arguments, generate a few thousand neighbouring inputs, run them through the gold version (library present, in a sidecar environment) and the agent's version, compare. This is DiffTGen and Opad with a canonical reference. It defeats the stub that hardcodes test inputs and the rewrite that is correct only on the inputs it was shown. The field calls this *differential fuzzing* and uses it to certify compilers, crypto libraries and browsers against each other.

G4c Extreme mutation of the replacement. For each function the agent added or modified, replace its body with a type-appropriate constant and rerun G2 and G4a. If both still pass, the function is pseudo-tested: nothing in the oracle constrains it. Report the fraction. A rewrite where most new functions are pseudo-tested has not been verified, regardless of what G2 says. This is Descartes applied to the patch rather than to the project.

The three layers compose. G4a is cheap and catches definite divergence on known inputs. G4b widens the input space. G4c tells you how much of the replacement the oracle actually constrains, so the grade carries a confidence rather than a bare pass.

G5 No phantom use

Diff declared layer against resolved layer. If a package the code imports is present only transitively, fail. Catches *phantom*.

G6 Quality not degraded

Deterministic only. Cyclomatic complexity, nesting depth and function length of changed functions, before and after, via `radon`. LLM-as-judge is excluded by design: SWE Atlas and RefactorPlatform use it, and the hacker-fixer paper shows rubric judges are gamed by length and verbosity. A deterministic metric is a weak signal but an honest one.

G7 Test oracle untouched (from Sushane)

Diff confined to test files with skip markers added, test functions removed, or assertions deleted fails. Sushane's finding matters here: skipping a test does not change the import graph, so this *cannot* be detected structurally and must inspect the test diff. Validated 79 of 80 on constructed variants.

G8 Closure not grown (from Sushane)

Resolved package set after the change must be a strict subset of before, and no new third-party package may appear in the manifest. Catches *trade one for three*. Validated 66 of 73.

What each cheat hits

Cheat	Passes G2?	Caught by
Hide in an extra	often	G1
Phantom (rely on transitive install)	yes	G5
Vendor, verbatim	yes	G3
Vendor, renamed identifiers	yes	G3 (winnowing)
Stub, trivial bodies	sometimes	G4a, G4c
Stub, filler logic, library name reused	sometimes	G4a
Stub, filler logic, internal module, no name reuse	sometimes	G4a, G4b
Hardcode test inputs	yes	G4b
Correct on tested inputs, wrong elsewhere	yes	G4b
Weaken tests	yes	G7
Trade one library for three	yes	G8

The bold row is the one Sushane showed no structural signal can catch. It falls to G4a and G4b because a filler function, however plausible its shape, does not return what the library returned.

Why this is defensible against an adaptive adversary

A grader is only as good as its resistance to an agent that knows the grader.

- The agent sees the test suite, so G2 is gameable. It does **not** see the carved recordings or the fuzz inputs; those are generated at grading time from the gold run. This is the held-out-tests remedy from SpecBench, obtained for free.
- G4b inputs are randomised per grading run, so there is no fixed set to overfit.
- G4c cannot be gamed by writing *more* code; a pseudo-tested function stays pseudo-tested no matter its length.
- The only way to pass G4 is to return what the library returns on inputs the agent has not seen. That is the definition of a correct removal.

We should still run the hacker-fixer loop against the full stack before claiming this, and report whatever it finds.

Honest limits, stated before a reviewer does

Equivalence is undecidable. G4 is sampling, not proof. A rewrite can diverge on an input neither the tests nor the fuzzer reached. We report confidence, never certainty, and G4c says how much of the replacement is actually constrained.

Recording fragility. Carving breaks on unserialisable state, nondeterminism, I/O and time. Elbaum's own work spends most of its effort here. Our usage sites are mostly pure data transformations (width calculation,

slugify, colour codes, TOML parsing), which is the easy case, but some will not record cleanly and the grader must say so rather than silently skip.

The sidecar. G4b needs the library present to compute reference outputs. That is a second environment per instance, roughly doubling grading cost for that layer. Affordable at our scale; the paper should quote the number.

Normalised equality is a judgement. Deciding that two floats are equal within 1e-9, or that dict ordering does not matter, encodes an opinion about what behaviour means. The rules must be published with the benchmark.

Not every removal is replayable. If the library did I/O, held network connections, or managed external state, carved replay does not apply. Those instances should be tiered separately and graded on G1 to G3 and G5 to G8 with an explicit "behaviour unverified" flag.

Novelty, stated carefully

Every component exists. Differential testing, carving, extreme mutation, winnowing, phantom detection are all published and some are decades old.

What does not exist is their composition for this task, and the observation that makes the composition possible: **in a removal task the removed library is a canonical oracle, which is the one thing every other repository-change benchmark lacks.** SWE-bench has a gold patch that may be wrong. Migration benchmarks have a target library whose semantics differ by design. Greenfield benchmarks have nothing. We have the exact artifact whose behaviour must be preserved, running, with recorded outputs. That is why a grader that is impossible in general becomes buildable here, and that is the sentence the paper should lead with.

Build order

1. **G7, G8:** already written by Sushane, integration only.
2. **G3 upgrade:** swap token-Jaccard for winnowing; extend to modified files.
3. **Phase 0 recorder + G4a:** the usage-site recorder and replay. This alone converts G4 from heuristic to behavioural and catches the `wcwidth` case.
4. **G4c:** extreme mutation of added functions; cheap once G4a exists, reuses the same replay.
5. **G4b:** Hypothesis strategies from recorded argument distributions; the sidecar environment.
6. **G6:** radon deltas.
7. **Adversarial hardening:** run the hacker-fixer loop; report survivors.

Validate each step against the 176 constructed cheats, Sushane's 597, and the 52 agent patches we already have. The pseudo-genuine family is the acceptance test: when G4a and G4b catch those 79, the grader is done.

Sources

- Smith, Barr, Le Goues, Brun. *Is the Cure Worse Than the Disease? Overfitting in Automated Program Repair*. FSE 2015.

- Xin, Reiss. *Identifying Test-Suite-Overfitted Patches through Test Case Generation* (DiffTGen). ISSTA 2017.
- Yang, Zhikhartsev, Liu, Tan. *Better Test Cases for Better Automated Program Repair* (Opad). FSE 2017.
- Xiong et al. *Identifying Patch Correctness in Test-Based Program Repair* (PATCH-SIM). ICSE 2018.
- Wang, Pradel, Liu. *PatchDiff: differential patch testing*. ICSE 2026, arXiv 2503.15223.
- [Patch Correctness Assessment: A Survey](#). TOSEM 2024.
- Elbaum, Chin, Dwyer, Jorde. *Carving and Replaying Differential Unit Test Cases from System Test Cases*. ICSE 2006 / TSE 2009.
- [Observation-based unit test generation at Meta](#). 2024.
- [Vera-Pérez, Danglot, Monperrus, Baudry. A Comprehensive Study of Pseudo-Tested Methods](#). EMSE 2019. Descartes tool.
- [Trail of Bits. Detecting code copying at scale with Vendetect](#). 2025. Winnowing, Schleimer et al. SIGMOD 2003.
- [Zhong et al. Hardening Agent Benchmarks with Adversarial Hacker-Fixer Loops](#). 2026.
- SpecBench, arXiv 2605.21384; ImpossibleBench, arXiv 2510.20270.
- Hypothesis property-based testing; differential fuzzing as surveyed in [FuzzDiff](#) and [MigrateLib](#).